

Concepts

Message Bus

The `MessageBus` enables communication between system components through message passing. This design creates a loosely coupled architecture where components interact without direct dependencies.

The *messaging patterns* include:

- Point-to-Point
- Publish/Subscribe
- Request/Response

Messages exchanged via the `MessageBus` fall into three categories:

- Data
- Events
- Commands

Message integrity

Once a message is created, its fields must not be mutated. This includes container fields such as `params` maps. Components can read a message and derive local state from it, but they must not rewrite the original.

Immutable messages keep every consumer seeing the same input, preserve what was true at emission time, and remove a class of shared-state races. Replay, debugging, and audit all depend on messages remaining stable after dispatch.

Three ownership rules follow from this:

- Caller-supplied request options stay on the message.

- Response metadata returned to the caller stays on the response.
- Component workflow state (bounded date ranges, grouping state, replay cursors, counters, processing flags) stays in component-owned context keyed by message or request ID.

When a component needs a derived message, it creates a new one with the required values instead of rewriting the original.

Data and signal publishing

While the `MessageBus` is a lower-level component that users typically interact with indirectly, `Actor` and `Strategy` classes provide convenient methods built on top of it:

```
def publish_data(self, data_type: DataType, data: Data) -> None:  
def publish_signal(self, name: str, value, ts_event: int = 0) -> None:
```



These methods allow you to publish custom data and signals efficiently without needing to work directly with the `MessageBus` interface.

Direct access

For advanced users or specialized use cases, direct access to the message bus is available within `Actor` and `Strategy` classes through the `self.msgbus` reference, which provides the full message bus interface.

To publish a custom message directly, you can specify a topic as a `str` and any Python `object` as the message payload, for example:

```
self.msgbus.publish("MyTopic", "MyMessage")
```



Messaging styles

NautilusTrader is an **event-driven** framework where components communicate by sending and receiving messages. Understanding the different messaging styles helps when building trading systems.

This guide explains the three primary messaging patterns available in NautilusTrader:

Messaging Style	Purpose	Best For
MessageBus - Publish/Subscribe to topics	Low-level, direct access to the message bus	Custom events, system-level communication
Actor-Based - Publish/Subscribe Data	Structured trading data exchange	Trading metrics, indicators, data needing persistence
Actor-Based - Publish/Subscribe Signal	Lightweight notifications	Simple alerts, flags, status updates

Each approach serves different purposes. This section helps you decide which pattern to use.

MessageBus publish/subscribe to topics

Concept

The `MessageBus` is the central hub for all messages in NautilusTrader. It enables a **publish/subscribe** pattern where components can publish events to **named topics**, and other components can subscribe to receive those messages. This decouples components, allowing them to interact indirectly via the message bus.

Key benefits and use cases

The message bus approach is ideal when you need:

- **Cross-component communication** within the system.
- **Flexibility** to define any topic and send any type of payload (any Python object).
- **Decoupling** between publishers and subscribers who don't need to know about each other.
- **Global Reach** where messages can be received by multiple subscribers.
- Working with events that don't fit within the predefined `Actor` model.
- Advanced scenarios requiring full control over messaging.

Considerations

- You must track topic names manually (typos could result in missed messages).
- You must define handlers manually.

Quick overview code

```
from nautilus_trader.core.message import Event

# Define a custom event
class Each10thBarEvent(Event):
    TOPIC = "each_10th_bar" # Topic name
    def __init__(self, bar):
        self.bar = bar

# Subscribe in a component (in Strategy)
self.msgbus.subscribe(Each10thBarEvent.TOPIC, self.on_each_10th_bar)

# Publish an event (in Strategy)
event = Each10thBarEvent(bar)
self.msgbus.publish(Each10thBarEvent.TOPIC, event)

# Handler (in Strategy)
def on_each_10th_bar(self, event: Each10thBarEvent):
    self.log.info(f"Received 10th bar: {event.bar}")
```

Full example

[MessageBus Example](#)

Actor-based publish/subscribe data

Concept

This approach provides a way to exchange trading specific data between **Actor**s in the system. (note: each **Strategy** inherits from **Actor**). It inherits from **Data**, which ensures proper timestamping and ordering of events - crucial for correct backtest processing.

Key benefits and use cases

The Data publish/subscribe approach works well when you need:

- **Exchange of structured trading data** like market data, indicators, custom metrics, or option greeks.
- **Proper event ordering** via built-in timestamps (**ts_event**, **ts_init**) crucial for backtest accuracy.
- **Data persistence and serialization** through the **@customdataclass** decorator, integrating with NautilusTrader's data catalog system.
- **Standardized trading data exchange** between system components.

Considerations

- Requires defining a class that inherits from `Data` or uses `@customdataclass`.

Inheriting from `Data` vs. using `@customdataclass`

Inheriting from `Data` class:

- Defines abstract properties `ts_event` and `ts_init` that must be implemented by the subclass. These ensure proper data ordering in backtests based on timestamps.

The `@customdataclass` decorator:

- Adds `ts_event` and `ts_init` attributes if they are not already present.
- Provides serialization functions: `to_dict()`, `from_dict()`, `to_bytes()`, `to_arrow()`, etc.
- Enables data persistence and external communication.

Quick overview code

```
from nautilus_trader.core.data import Data
from nautilus_trader.model.custom import customdataclass

@customdataclass
class GreeksData(Data):
    delta: float
    gamma: float

# Publish data (in Actor / Strategy)
data = GreeksData(delta=0.75, gamma=0.1, ts_event=1_630_000_000_000_000, ts_init=1_630_000_000_000_000)
self.publish_data(GreeksData, data)

# Subscribe to receiving data (in Actor / Strategy)
self.subscribe_data(GreeksData)

# Handler (this is static callback function with fixed name)
def on_data(self, data: Data):
    if isinstance(data, GreeksData):
        self.log.info(f"Delta: {data.delta}, Gamma: {data.gamma}")
```

Full example

[Actor-Based Data Example](#)

Actor-based publish/subscribe signal

Concept

Signals are a lightweight way to publish and subscribe to simple notifications within the actor framework. This is the simplest messaging approach, requiring no custom class definitions.

Key benefits and use cases

The Signal messaging approach works well when you need:

- Simple, lightweight notifications/alerts like "RiskThresholdExceeded" or "TrendUp".
- Quick, on-the-fly messaging without defining custom classes.
- Broadcasting alerts or flags as primitive data (`int`, `float`, or `str`).
- Easy API integration with straightforward methods (`publish_signal`, `subscribe_signal`).
- Multiple subscriber communication where all subscribers receive signals when published.
- Minimal setup overhead with no class definitions required.

Considerations

- Each signal can contain only **single value** of type: `int`, `float`, and `str`. That means no support for complex data structures or other Python types.
- In the `on_signal` handler, you can only differentiate between signals using `signal.value`, as the signal name is not accessible in the handler.

Quick overview code

```
# Define signal constants for better organization (optional but recommended)
import types
from nautilus_trader.core.datetime import unix_nanos_to_dt
from nautilus_trader.common.enums import LogColor

signals = types.SimpleNamespace()
signals.NEW_HIGHEST_PRICE = "NewHighestPriceReached"
signals.NEW_LOWEST_PRICE = "NewLowestPriceReached"

# Subscribe to signals (in Actor/Strategy)
self.subscribe_signal(signals.NEW_HIGHEST_PRICE)
self.subscribe_signal(signals.NEW_LOWEST_PRICE)

# Publish a signal (in Actor/Strategy)
self.publish_signal(
    name=signals.NEW_HIGHEST_PRICE,
    value=signals.NEW_HIGHEST_PRICE, # value can be the same as name for simplicity
    ts_event=bar.ts_event, # timestamp from triggering event
)

# Handler (this is static callback function with fixed name)
def on_signal(self, signal):
```

```
# IMPORTANT: We match against signal.value, not signal.name
match signal.value:
    case signals.NEW_HIGHEST_PRICE:
        self.log.info(
            f"New highest price was reached. | "
            f"Signal value: {signal.value} | "
            f"Signal time: {unix_nanos_to_dt(signal.ts_event)}",
            color=LogColor.GREEN
        )
```

Full example

[Actor-Based Signal Example](#)

Summary and decision guide

Here's a quick reference to help you decide which messaging style to use:

Decision guide: Which style to choose?

Use Case	Recommended Approach	Setup required
Custom events or system-level communication	<code>MessageBus</code> + Pub/Sub to topic	Topic + Handler management
Structured trading data	<code>Actor</code> + Pub/Sub Data + optional <code>@customdataclass</code> if serialization is needed	New class definition inheriting from <code>Data</code> (handler <code>on_data</code> is predefined)
Simple alerts/notifications	<code>Actor</code> + Pub/Sub Signal	Signal name only

External publishing

The `MessageBus` can be *backed* with any database or message broker technology which has an integration written for it, this then enables external publishing of messages.

i Redis is currently supported for all serializable messages which are published externally. The minimum supported Redis version is 6.2 (required for [streams](#) functionality).

Under the hood, when a backing database (or any other compatible technology) is configured, all outgoing messages are first serialized, then transmitted via a Multiple-Producer Single-Consumer (MPSC) channel to a separate thread (implemented in Rust). In

this separate thread, the message is written to its final destination, which is presently Redis streams.

Offloading I/O to a separate thread keeps the main thread unblocked.

Serialization

Nautilus supports serialization for:

- All Nautilus built-in types (serialized as dictionaries `dict[str, Any]` containing serializable primitives).
- Python primitive types (`str`, `int`, `float`, `bool`, `bytes`).

You can add serialization support for custom types by registering them through the `serialization` subpackage.

```
def register_serializable_type(
    cls,
    to_dict: Callable[[Any], dict[str, Any]],
    from_dict: Callable[[dict[str, Any]], Any],
):
    ...
```

- `cls`: The type to register.
- `to_dict`: The delegate to instantiate a dict of primitive types from the object.
- `from_dict`: The delegate to instantiate the object from a dict of primitive types.

Configuration

The message bus external backing technology can be configured by importing the `MessageBusConfig` object and passing this to your `TradingNodeConfig`. Each of these config options will be described below.

```
... # Other config omitted
message_bus=MessageBusConfig(
    database=DatabaseConfig(),
    encoding="json",
    timestamps_as_iso8601=True,
    buffer_interval_ms=100,
    autotrim_mins=30,
```

```

use_trader_prefix=True,
use_trader_id=True,
use_instance_id=False,
streams_prefix="streams",
types_filter=[QuoteTick, TradeTick],
)
...

```

Database config

A `DatabaseConfig` must be provided, for a default Redis setup on the local loopback you can pass a `DatabaseConfig()`, which will use defaults to match.

Encoding

Two encodings are currently supported by the built-in `Serializer` used by the `MessageBus`:

- JSON (`json`)
- MessagePack (`msgpack`)

Use the `encoding` config option to control the message writing encoding.



The `msgpack` encoding is used by default as it offers the most optimal serialization and memory performance. We recommend using `json` encoding for human readability when performance is not a primary concern.

Timestamp formatting

By default timestamps are formatted as UNIX epoch nanosecond integers. Alternatively you can configure ISO 8601 string formatting by setting the `timestamps_as_iso8601` to `True`.

Message stream keys

Message stream keys are essential for identifying individual trader nodes and organizing messages within streams. They can be tailored to meet your specific requirements and use cases. In the context of message bus streams, a trader key is typically structured as follows:

```
trader:{trader_id}:{instance_id}:{streams_prefix}
```



The following options are available for configuring message stream keys:

Trader prefix

If the key should begin with the `trader` string.

Trader ID

If the key should include the trader ID for the node.

Instance ID

Each trader node is assigned a unique 'instance ID,' which is a UUIDv4. This instance ID helps distinguish individual traders when messages are distributed across multiple streams. You can include the instance ID in the trader key by setting the `use_instance_id` configuration option to `True`. This is particularly useful when you need to track and identify traders across various streams in a multi-node trading system.

Streams prefix

The `streams_prefix` string enables you to group all streams for a single trader instance or organize messages for multiple instances. Configure this by passing a string to the `streams_prefix` configuration option, ensuring other prefixes are set to false.

Stream per topic

Indicates whether the producer will write a separate stream for each topic. This is particularly useful for Redis backings, which do not support wildcard topics when listening to streams. If set to False, all messages will be written to the same stream.

 Redis does not support wildcard stream topics. For better compatibility with Redis, it is recommended to set this option to False.

Types filtering

When messages are published on the message bus, they are serialized and written to a stream if a backing for the message bus is configured and enabled. To prevent flooding the stream with data like high-frequency quotes, you may filter out certain types of messages from external publication.

To enable this filtering mechanism, pass a list of `type` objects to the `types_filter` parameter in the message bus configuration, specifying which types of messages should be excluded from external publication.

```
from nautilus_trader.config import MessageBusConfig
from nautilus_trader.model.data import QuoteTick
from nautilus_trader.model.data import TradeTick

# Create a MessageBusConfig instance with types filtering
message_bus = MessageBusConfig(
    types_filter=[QuoteTick, TradeTick]
)
```



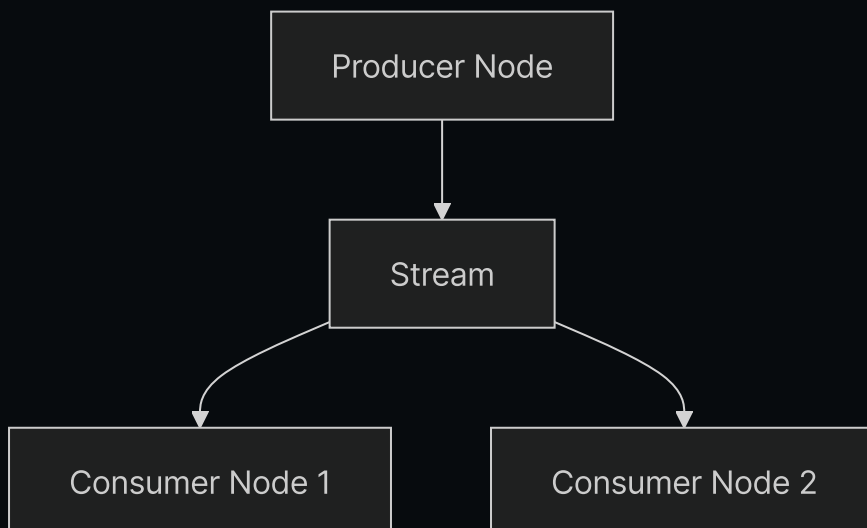
Stream auto-trimming

The `autotrim_mins` configuration parameter allows you to specify the lookback window in minutes for automatic stream trimming in your message streams. Automatic stream trimming helps manage the size of your message streams by removing older messages, ensuring that the streams remain manageable in terms of storage and performance.

i The current Redis implementation will maintain the `autotrim_mins` as a maximum width (plus roughly a minute, as streams are trimmed no more than once per minute). Rather than a maximum lookback window based on the current wall clock time.

External streams

The message bus within a `TradingNode` (node) is referred to as the "internal message bus". A producer node is one which publishes messages onto an external stream (see [external publishing](#)). The consumer node listens to external streams to receive and publish deserialized message payloads on its internal message bus.





Set the `LiveDataEngineConfig.external_clients` with the list of `client_id`s intended to represent the external streaming clients. The `DataEngine` will filter out subscription commands for these clients, ensuring that the external streaming provides the necessary data for any subscriptions to these clients.

Example configuration

The following example details a streaming setup where a producer node publishes Binance data externally, and a downstream consumer node publishes these data messages onto its internal message bus.

Producer node

We configure the `MessageBus` of the producer node to publish to a `"binance"` stream. The settings `use_trader_id`, `use_trader_prefix`, and `use_instance_id` are all set to `False` to ensure a simple and predictable stream key that the consumer nodes can register for.

```
message_bus=MessageBusConfig(  
    database=DatabaseConfig(  
        connection_timeout=2,  
        response_timeout=2,  
    ),  
    use_trader_id=False,  
    use_trader_prefix=False,  
    use_instance_id=False,  
    streams_prefix="binance", # <---  
    stream_per_topic=False,  
    autotrim_mins=30,  
)
```



Consumer node

We configure the `MessageBus` of the consumer node to receive messages from the same `"binance"` stream. The node will listen to the external stream keys to publish these messages onto its internal message bus. Additionally, we declare the client ID `"BINANCE_EXT"` as an external client. This ensures that the `DataEngine` does not attempt to send data commands to this client ID, as we expect these messages to be published onto the internal message bus from the external stream, to which the node has subscribed to the relevant topics.

```
data_engine=LiveDataEngineConfig(  
    external_clients=[ClientId("BINANCE_EXT")],  
)  
message_bus=MessageBusConfig(  
    database=DatabaseConfig(  

```



```
        connection_timeout=2,  
        response_timeout=2,  
    ),  
    external_streams=["binance"], # <---  
),
```

Related guides

- [Actors](#) - Actors use the message bus for event handling.
- [Architecture](#) - Message bus role in system architecture.

[< Cache](#)[Previous Page](#)[Accounting >](#)[Next Page](#)